

Complete Coding Pattern Reference.....	3
How to use this document.....	3
Quick Reference Table.....	3
1. Two Pointers ● Must-know.....	5
Pattern 1 — Converging (opposite ends).....	5
Pattern 2 — Same-Direction (slow build + fast scan).....	5
What to Write Tonight.....	5
Fast Mental Triage.....	5
2. Sliding Window ● Must-know.....	6
Pattern 1 — Fixed-Size Window.....	6
Pattern 2 — Variable-Size Window.....	6
What to Write Tonight.....	6
Fast Mental Triage.....	6
3. Prefix Sum / HashMap Frequency ● Must-know.....	7
Pattern 1 — Prefix Sum Precompute.....	7
Pattern 2 — HashMap/HashSet Frequency Lookup.....	7
What to Write Tonight.....	7
Fast Mental Triage.....	7
4. Binary Search ● Must-know.....	8
Pattern 1 — Classic Exact Match.....	8
Pattern 2 — Rotated/Modified Array.....	8
Pattern 3 — Search on the Answer.....	8
What to Write Tonight.....	9
Fast Mental Triage.....	9
5. Intervals ● Must-know.....	9
Pattern 1 — Sort + Merge.....	9
Pattern 2 — Sort + Greedy (remove minimum).....	10
Pattern 3 — Sort + Min-Heap (track concurrent events).....	10
What to Write Tonight.....	11
Fast Mental Triage.....	11
6. Fast & Slow Pointers (Linked List) ● Important.....	11
Write tonight.....	12
Triage.....	12
7. Tree DFS/BFS ● Must-know.....	12
Pattern 1 — Bottom-Up Recursive (combine children's results).....	12
Pattern 2 — Top-Down with Passed State.....	12
Pattern 3 — Level-Order BFS.....	13
Pattern 4 — BST Ordering Exploitation.....	13
Pattern 5 — Serialization (pre-order + markers).....	13
What to Write Tonight.....	14
Fast Mental Triage.....	14
8. Graphs ● Must-know.....	14
Pattern 1 — BFS (Breadth-First Search).....	14

Pattern 2 — DFS (Depth-First Search).....	15
Pattern 3 — Topological Sort.....	15
Pattern 4 — Union-Find (Disjoint Set).....	16
What to Actually Write Tonight.....	17
The Fast Mental Triage.....	17
9. Tries ● Important.....	17
Write tonight.....	18
Triage.....	18
10. 1D Dynamic Programming ● Must-know.....	18
Pattern 1 — Fibonacci-style (depends on last 1-2 states).....	18
Pattern 2 — Linear Decision (take or skip).....	18
Pattern 3 — Kadane's (running max with reset).....	19
Pattern 4 — LIS-style (compare against ALL previous indices).....	19
What to Actually Write Tonight.....	20
The Fast Mental Triage in an Interview.....	20
11. 2D Dynamic Programming ● Good-to-know.....	20
Pattern 1 — Sequence Alignment.....	20
Pattern 2 — Grid Path Counting.....	20
What to Write Tonight.....	21
Fast Mental Triage.....	21
12. Backtracking ● Important.....	21
Pattern 1 — Subset/Combination Building.....	21
Pattern 2 — Permutation Building.....	21
Pattern 3 — Grid Path Search.....	22
What to Write Tonight.....	22
Fast Mental Triage.....	22
13. Heap / Top-K ● Important.....	22
Pattern 1 — Single Size-K Heap.....	22
Pattern 2 — Two Balanced Heaps.....	23
Pattern 3 — K-Way Merge.....	23
Pattern 4 — Greedy Heap Simulation.....	23
What to Write Tonight.....	23
Fast Mental Triage.....	23
14. Stack-Based Patterns ● Important.....	24
Pattern 1 — Matching/Resolution Stack.....	24
Pattern 2 — Monotonic Stack/Queue NEW	24
Write tonight.....	25
Triage.....	25
15. Matrix Traversal / In-Place Manipulation ● Good-to-know.....	25
Pattern 1 — Boundary-Shrinking Traversal.....	25
Pattern 2 — Transpose + Reverse / In-Place Marker Reuse.....	26
What to Write Tonight.....	26
Fast Mental Triage.....	26
16. Bit Manipulation ● Good-to-know.....	26

Write tonight.....	27
Triage.....	27
17. Design a Data Structure ● Must-know.....	27
Pattern 1 — HashMap + Doubly Linked List (order by recency).....	27
Pattern 2 — HashMap + Frequency Buckets (order by count).....	29
Pattern 3 — HashMap + Swap-and-Pop Array (O(1) arbitrary removal).....	29
Pattern 4 — HashMap + Sorted Append List + Binary Search (order by time).....	30
Combination callout — Design Twitter.....	30
What to Write Tonight.....	31
Fast Mental Triage.....	31
Closing Note.....	31

Complete Coding Pattern Reference

This is the full map of every pattern underlying Blind 75 plus the Google-tagged extras in your revision pool — 17 categories total. Nothing in LeetCode-style interview coding sits outside these.

● **Must-know** — will almost certainly appear · ● **Important** — very common · ● **Good-to-know** — lower frequency but still worth having.

How to use this document

Don't read it cover to cover in one sitting. Pull 1-2 sections per evening alongside whatever's already scheduled. For each pattern: read the trigger words, trace through the example, then close the document and write the "What to Write Tonight" block from memory. That act of retrieval — not the reading — is what actually builds recognition speed for an interview.

Quick Reference Table

#	Category	Priority	One-line trigger
1	Two Pointers	●	Sorted array + pair/triplet condition, or palindrome check
2	Sliding Window	●	Longest/shortest contiguous range satisfying a condition
3	Prefix Sum / HashMap Frequency	●	Subarray sum queries, or "have I seen this"

4	Binary Search	●	Sorted/rotated array, or "minimize/maximize X such that..."
5	Intervals	●	Merge, remove-minimum, or count-concurrent on ranges
6	Fast & Slow Pointers	●	Cycle detection, middle of list, Nth-from-end
7	Tree DFS/BFS	●	Any binary tree problem
8	Graphs	●	Connectivity, shortest path, ordering with dependencies
9	Tries	●	Prefix matching, autocomplete, dictionary lookup
10	1D Dynamic Programming	●	Optimal value built up from a 1D sequence
11	2D Dynamic Programming	●	Two sequences compared, or grid path counting
12	Backtracking	●	"Generate ALL X" — combinations, permutations, grid paths
13	Heap / Top-K	●	"K largest/smallest," running median, merge K sorted
14	Stack-Based Patterns	●	Matching pairs, or "next greater/smaller element"
15	Matrix Traversal	●	Spiral order, rotate, in-place grid transform
16	Bit Manipulation	●	Binary representation, "without +/-"
17	Design a Data Structure	●	"Build a class supporting X in $O(1)/O(\log n)$ "

1. Two Pointers Must-know

Pattern 1 — Converging (opposite ends)

Logic: Start at both ends, move inward based on a comparison.

Trigger words: sorted array + pair/triplet sum, or palindrome check.

Example: Container With Most Water — always move the *shorter* wall inward, since the taller one can't be the bottleneck.

```
left, right = 0, len(arr) - 1
while left < right:
    if condition(arr[left], arr[right]): record()
    elif need_bigger: left += 1
    else: right -= 1
```

Pattern 2 — Same-Direction (slow build + fast scan)

Logic: One pointer scans ahead; the other only advances when a valid element is found, effectively "compacting" the array in place.

Trigger words: "remove/move elements in place," partitioning by a condition.

```
slow = 0
for fast in range(len(arr)):
    if keep_condition(arr[fast]):
        arr[slow] = arr[fast]
        slow += 1
```

What to Write Tonight

1. Converging: start both ends, move inward | pair sum / palindrome
2. Same-direction: slow writes, fast scans | remove/partition in place

Fast Mental Triage

Comparing from outside-in (sum/palindrome) → converging. Compacting or filtering a single array in place → same-direction.

2. Sliding Window Must-know

Pattern 1 — Fixed-Size Window

Logic: Window size is given upfront; slide it one step at a time, updating the running total/state incrementally instead of recomputing.

Trigger words: "subarray/substring of size exactly K."

```
window_sum = sum(arr[:k])
best = window_sum
for i in range(k, len(arr)):
    window_sum += arr[i] - arr[i-k]
    best = max(best, window_sum)
```

Pattern 2 — Variable-Size Window

Logic: Expand right to grow; shrink left when invalid; track the best window seen.

Trigger words: "longest/shortest substring/subarray satisfying condition X" — no fixed size given.

Example: Longest Substring Without Repeating Characters; Minimum Window Substring.

```
left = 0
for right in range(len(s)):
    update_window(s[right])
    while window_invalid():
        remove_from_window(s[left])
        left += 1
    update_best(left, right)
```

What to Write Tonight

1. Fixed-size: slide by 1, $O(1)$ update per step | "subarray of size K"
2. Variable-size: expand right, shrink left on invalid | "longest/shortest with condition"

Fast Mental Triage

Is a window size literally given as a number in the problem? → fixed-size. Otherwise, optimizing length under some condition? → variable-size, by far the more common of the two on Blind 75.

3. Prefix Sum / HashMap Frequency

Must-know

Pattern 1 — Prefix Sum Precompute

Logic: Compute cumulative sums once; any range-sum query becomes $O(1)$ subtraction.

Trigger words: "subarray sum equals X," repeated range-sum queries.

Example: Product of Array Except Self (prefix $\times$ suffix products); Range Sum Query 2D.

```
prefix = [0] * (len(arr) + 1)
for i in range(len(arr)):
    prefix[i+1] = prefix[i] + arr[i]
# sum(i..j inclusive) = prefix[j+1] - prefix[i]
```

Pattern 2 — HashMap/HashSet Frequency Lookup

Logic: $O(1)$ membership/count checks instead of a linear re-scan.

Trigger words: "have I seen this," "same signature/anagram," "consecutive sequence."

Example: Longest Consecutive Sequence — only start counting from numbers where $n-1$ isn't in the set, so each run is counted exactly once.

```
seen = set(arr)
for n in arr:
    if n - 1 not in seen: # start of a run
        length = 1
        while n + length in seen:
            length += 1
```

What to Write Tonight

1. Prefix sum: precompute cumulative array, $O(1)$ range queries | "subarray sum X"
2. HashMap/Set: $O(1)$ membership or frequency count | "seen before" / "same signature"

Fast Mental Triage

Numeric range/sum queries repeated many times $\rightarrow$ prefix sum. Checking existence, counting occurrences, or comparing structural signatures $\rightarrow$ HashMap/Set. Both replace an $O(n)$ re-scan with $O(1)$ lookups — the difference is just *what* you're precomputing.

4. Binary Search Must-know

Pattern 1 — Classic Exact Match

Logic: Halve the search space; compare midpoint to target.

Trigger words: sorted array, find exact value or insertion point.

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Pattern 2 — Rotated/Modified Array

Logic: Determine which half is "normally sorted" at each step, then decide if the target could be in it.

Trigger words: "rotated sorted array."

Example: Search in Rotated Sorted Array; Find Minimum in Rotated Sorted Array.

```
if arr[left] <= arr[mid]:          # left half sorted
    if arr[left] <= target < arr[mid]: right = mid - 1
    else: left = mid + 1
else:                             # right half sorted
    if arr[mid] < target <= arr[right]: left = mid + 1
    else: right = mid - 1
```

Pattern 3 — Search on the Answer

Logic: Binary search over a **range of possible answers**, not array indices — works whenever "can we achieve X?" is a monotonic yes/no function.

Trigger words: "minimize the maximum," "find the smallest/largest value such that condition holds" — no array indexing language at all.

Example: Median of Two Sorted Arrays (binary search the partition point, not a value).

```
lo, hi = min_possible_answer, max_possible_answer
while lo < hi:
    mid = (lo + hi) // 2
    if can_achieve(mid):
        hi = mid
    else:
        lo = mid + 1
```

What to Write Tonight

1. Classic: compare mid to target directly | sorted array, find value
2. Rotated: find which half is sorted, then decide | "rotated sorted array"
3. On-answer: binary search a value range, not index | "minimize/maximize X s.t. condition"

Fast Mental Triage

Plain sorted array → classic. "Rotated" in the problem text → Pattern 2. No array at all, just "what's the best achievable value" → Pattern 3 — this is the one people miss because it doesn't *look* like binary search.

5. Intervals Must-know

Pattern 1 — Sort + Merge

Logic: Sort by start time, then walk through and merge any interval that overlaps with the last one you kept.

Trigger words: "merge," "combine overlapping intervals," "insert and merge."

Example: Merge Intervals — sort by start; if current interval's start $\leq$ previous interval's end, extend the previous interval's end; otherwise add it as a new interval.

```
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    result = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= result[-1][1]:
            result[-1][1] = max(result[-1][1], end)
```

```

    else:
        result.append([start, end])
return result

```

Pattern 2 — Sort + Greedy (remove minimum)

Logic: Sort by **end** time (not start — this is the key difference from Pattern 1), then greedily keep whichever interval ends earliest, discarding anything that conflicts with it.

Trigger words: "minimum removals," "non-overlapping," "erase minimum number of intervals."

Example: Non-overlapping Intervals — sort by end time; keep an interval if its start is $\geq$ the end of the last kept interval; otherwise it must be removed.

```

def erase_overlap_intervals(intervals):
    intervals.sort(key=lambda x: x[1])
    last_end = float("-inf")
    removals = 0
    for start, end in intervals:
        if start >= last_end:
            last_end = end      # keep it
        else:
            removals += 1      # must remove it
    return removals

```

Sorting by **end** rather than **start** is the detail people forget — it's what guarantees the greedy choice is optimal (keeping the earliest-ending interval always leaves the most room for future intervals).

Pattern 3 — Sort + Min-Heap (track concurrent events)

Logic: Sort by start time; use a min-heap of active end times to track how many intervals are "in progress" at once.

Trigger words: "minimum rooms needed," "maximum concurrent events," "how many at the same time."

Example: Meeting Rooms II — sort by start; for each meeting, if the earliest-ending active meeting (heap top) has already finished ($\leq$ current start), reuse that room (pop it); otherwise push a new end time. Final heap size = rooms needed.

```

import heapq

```

```

def min_meeting_rooms(intervals):
    intervals.sort(key=lambda x: x[0])

```

```

heap = [] # min-heap of end times
for start, end in intervals:
    if heap and heap[0] <= start:
        heapq.heappop(heap) # reuse a room
    heapq.heappush(heap, end)
return len(heap)

```

What to Write Tonight

1. Sort+Merge: sort by START, merge if overlap | "combine intervals"
2. Sort+Greedy: sort by END, keep earliest-ending | "minimum removal"
3. Sort+MinHeap: sort by START, heap tracks active ends | "concurrent / rooms needed"

Fast Mental Triage

Does it ask you to combine/consolidate overlapping ranges into fewer ranges? → Pattern 1.
 Does it ask for the minimum number to remove/keep to eliminate all overlaps? → Pattern 2 (sort by end — not start). Does it ask how many overlap at once or minimum resources needed? → Pattern 3 (heap). The single biggest tell: Pattern 1 and 3 both sort by start, Pattern 2 sorts by end — mixing that up breaks the greedy logic immediately.

6. Fast & Slow Pointers (Linked List) 🟡

Important

Logic: Slow moves 1 step, fast moves 2. Used for cycle detection, finding the middle, or "Nth from the end" without knowing the list length up front. This is genuinely one mechanism, not several sub-patterns — cycle detection, middle-finding, and Nth-from-end are different *goals* using the exact same technique.

Trigger words: "detect cycle," "find the middle," "Nth node from the end," "find duplicate using O(1) space."

Examples:

- **Linked List Cycle (Floyd's)** — `slow = slow.next, fast = fast.next.next;` if they ever meet, a cycle exists.
- **Remove Nth Node From End** — advance fast `n` steps first, then move both together until fast hits the end; slow is now right before the target.
- **Reorder List** — find the middle with fast/slow, reverse the second half, then merge the two halves alternately.
- **Find the Duplicate Number** — treat array values as "next pointers" and run Floyd's cycle detection on that implicit linked structure.

```

slow = fast = head
while fast and fast.next:

```

```
slow = slow.next
fast = fast.next.next
if slow == fast:
    cycle_found()
```

Write tonight

Fast & Slow: slow+1, fast+2 | "cycle / middle / Nth from end"

Triage

Any time you'd otherwise need the list's length first (middle, Nth-from-end) or need cycle detection with O(1) memory → fast/slow solves it in one pass.

7. Tree DFS/BFS Must-know

Pattern 1 — Bottom-Up Recursive (combine children's results)

Logic: Recurse into both children first, then combine their return values to answer for the current node.

Trigger: depth, balance, sum, or any "answer depends on what came from below."

Example: Maximum Depth — `1 + max(depth(left), depth(right))`.

```
def dfs(node):
    if not node: return 0
    return 1 + max(dfs(node.left), dfs(node.right))
```

Pattern 2 — Top-Down with Passed State

Logic: Pass constraints/context DOWN through recursion calls, rather than building an answer UP.

Trigger: "must satisfy a range/bound inherited from ancestors" — classically BST validation.

Example: Validate BST — each node must fit within `(min, max)` bounds inherited from its parent.

```
def dfs(node, low, high):
```

```
if not node: return True
if not (low < node.val < high): return False
return dfs(node.left, low, node.val) and dfs(node.right, node.val, high)
```

Pattern 3 — Level-Order BFS

Logic: Queue-based traversal, processing one full level at a time using a size snapshot.

Trigger: "level by level," "level order," shortest path in an unweighted tree.

Example: Binary Tree Level Order Traversal.

```
from collections import deque
queue = deque([root])
while queue:
    level_size = len(queue)
    level = []
    for _ in range(level_size):
        node = queue.popleft()
        level.append(node.val)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
    result.append(level)
```

Pattern 4 — BST Ordering Exploitation

Logic: Use the $\text{left} < \text{node} < \text{right}$ property to skip entire subtrees instead of visiting everything.

Trigger: problem explicitly says BST (not just "binary tree") and asks something ordering-related.

Example: LCA of a BST — both targets less than current → go left; both greater → go right; otherwise current node is the answer.

```
def lca(node, p, q):
    if p.val < node.val and q.val < node.val: return lca(node.left, p, q)
    if p.val > node.val and q.val > node.val: return lca(node.right, p, q)
    return node
```

Pattern 5 — Serialization (pre-order + markers)

Logic: Walk pre-order, recording explicit null markers; rebuild by consuming tokens in the same order.

Trigger: "serialize," "encode/decode," "reconstruct from a traversal."

Example: Serialize and Deserialize Binary Tree; Construct from Preorder and Inorder (the latter uses `preorder[0]` as root, finds it in `inorder` to split subtrees).

What to Write Tonight

1. Bottom-up: `combine(dfs(left), dfs(right))` | depth/sum/balance
2. Top-down: `pass bounds/state down through calls` | BST validation
3. Level BFS: `queue + size-snapshot per level` | "level by level"
4. BST exploit: `compare to node, go left OR right only` | BST-specific ordering
5. Serialize: `pre-order + null markers` | encode/decode/reconstruct

Fast Mental Triage

Does the answer at a node depend on its **children's** answers? → bottom-up. Does it depend on **constraints from ancestors**? → top-down. "Level by level"? → BFS. Says **BST** specifically? → exploit ordering, don't brute-force both sides. Reconstructing from a traversal string/array? → serialization logic.

8. Graphs Must-know

Pattern 1 — BFS (Breadth-First Search)

Logic: Explore level by level using a queue — visit all neighbors at distance 1, then all at distance 2, and so on.

Trigger words: "shortest path" (unweighted graph), "minimum number of steps," "level order," "nearest."

Example: Shortest Path in Binary Matrix — BFS guarantees the first time you reach the target, it's via the shortest possible path, since you exhaust every shorter-distance node first.

```
queue = [start]
visited = {start}
while queue:
    node = queue.pop(0)
    for neighbor in node.neighbors:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)
```

BFS only gives correct shortest-path guarantees on **unweighted** graphs. The moment edges have different weights, you need Dijkstra instead.

Pattern 2 — DFS (Depth-First Search)

Logic: Go as deep as possible down one path before backtracking, using a stack (explicit or via recursion).

Trigger words: "explore all paths," "connected components," "cycle detection," "backtracking," "does a path exist."

Example: Number of Islands — DFS from each unvisited land cell, marking everything connected to it as visited, before moving to the next island.

```
def dfs(node, visited):
    visited.add(node)
    for neighbor in node.neighbors:
        if neighbor not in visited:
            dfs(neighbor, visited)
```

DFS is the natural fit whenever you need to backtrack — which is why it's the foundation for both cycle detection and topological sort.

Pattern 3 — Topological Sort

Logic: Order nodes in a directed graph such that every edge points from an earlier node to a later one. Only possible if the graph has **no cycle**.

Trigger words: "build order," "course prerequisites," "task scheduling with dependencies," "alphabetical/character ordering from pairwise constraints."

Example: Course Schedule and Alien Dictionary — both are really "can these dependencies be satisfied, and if so, in what order?"

Kahn's algorithm (BFS-based):

```
from collections import deque
```

```
def topo_sort(num_nodes, edges):
    graph = {i: [] for i in range(num_nodes)}
    in_degree = {i: 0 for i in range(num_nodes)}
    for u, v in edges:
        graph[u].append(v)
        in_degree[v] += 1

    queue = deque([n for n in in_degree if in_degree[n] == 0])
    result = []
```

```

while queue:
    node = queue.popleft()
    result.append(node)
    for neighbor in graph[node]:
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0:
            queue.append(neighbor)

if len(result) < num_nodes:
    return None # cycle exists, no valid ordering
return result

```

DFS-based: do a DFS, append each node to the result *after* recursing into all its neighbors (post-order), then reverse the result.

Kahn's is usually easier to reason about and naturally detects cycles — that's the version worth defaulting to in an interview.

Pattern 4 — Union-Find (Disjoint Set)

Logic: Track which nodes belong to the same connected group, supporting two operations efficiently: **union** (merge two groups) and **find** (check which group a node belongs to).

Trigger words: "connected components" (especially with incremental edges), "redundant connection," "are these in the same group," "minimum spanning tree" (Kruskal's).

Example: Number of Connected Components, Graph Valid Tree — both ask "how many separate groups exist" or "does adding this edge create a cycle."

```
parent = {node: node for node in nodes}
```

```

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x]) # path compression
    return parent[x]

```

```

def union(x, y):
    rootX, rootY = find(x), find(y)
    if rootX != rootY:
        parent[rootX] = rootY

```

The key tell for choosing Union-Find over DFS: when edges are being added **incrementally** (one at a time, and you need an answer after each addition), Union-Find updates in near-constant time per operation, while re-running DFS from scratch every time would be far more expensive.

What to Actually Write Tonight

1. BFS: queue, level-by-level | "shortest path" (unweighted)
2. DFS: stack/recursion, go deep | "all paths" / "connected components" / "cycles"
3. Topological sort: Kahn's BFS, in-degree=0 start | "build order" / "prerequisites"
4. Union-Find: parent pointers + path compress | "incremental connectivity" / "redundant edge"

The Fast Mental Triage

Need the **shortest** path and edges are unweighted? → BFS. Need to explore **everything reachable**, detect a **cycle**, or **backtrack**? → DFS. Need a valid **ordering** that respects dependencies? → Topological sort (and if it fails, that itself tells you a cycle exists). Are edges arriving **one at a time** and you keep needing "are these connected now"? → Union-Find.

9. Tries Important

Logic: A tree where each node is one character; a root-to-marked-node path spells a word — built for fast prefix operations. This is one data structure, walked in different ways for different problems, not multiple distinct algorithms.

Trigger words: prefix matching, autocomplete, "does any word start with...", large dictionary lookups.

Examples:

- **Implement Trie** — `children` map + `is_end` flag per node; insert/search/startsWith all just walk the tree.
- **Design Add and Search Words** — same Trie, but a wildcard `.` means "recurse into ALL children" instead of just one.
- **Word Search II** — build a Trie from the whole word list first, then DFS the grid while walking the Trie in parallel, abandoning a path the instant no word could match.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False
```

```
class Trie:
    def __init__(self):
        self.root = TrieNode()
    def insert(self, word):
```

```
node = self.root
for ch in word:
    node = node.children.setdefault(ch, TrieNode())
node.is_end = True
```

Write tonight

Tries: tree of chars, is_end flag | "prefix matching" / "autocomplete" / "dictionary word search"

Triage

Checking membership against a *large* dictionary repeatedly, or specifically needing prefix info (not just exact match) → Trie beats a HashSet.

10. 1D Dynamic Programming

Must-know

Pattern 1 — Fibonacci-style (depends on last 1-2 states)

Recurrence: $dp[i] = dp[i-1] + dp[i-2]$

Trigger words: "number of ways to...", "how many distinct ways," simple counting where each step only depends on a fixed small window behind it.

Example: Climbing Stairs — $dp[i]$ = ways to reach step i = ways from step $i-1$ (one step) + ways from step $i-2$ (two steps).

```
dp[0]=1; dp[1]=1
dp[i] = dp[i-1] + dp[i-2]
```

Pattern 2 — Linear Decision (take or skip)

Recurrence: $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$

Trigger words: "maximum/minimum value where you can't pick adjacent elements," constraint-based selection.

Example: House Robber — either skip house i (keep $dp[i-1]$) or rob house i (add $\text{nums}[i]$ to $dp[i-2]$, since you can't also rob $i-1$).

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i])
```

The key insight: at each index you're making a binary choice (include vs exclude), and that choice has a ripple effect on which earlier states are still "available."

Pattern 3 — Kadane's (running max with reset)

Recurrence: $dp[i] = \max(nums[i], dp[i-1] + nums[i])$

Trigger words: "maximum subarray sum," "best contiguous segment."

Example: Maximum Subarray — at each index, decide: extend the previous running subarray, or abandon it and start fresh from here (this happens when the running sum has gone negative — carrying it forward would only hurt).

```
def max_subarray(nums):
    dp = [0] * len(nums)
    dp[0] = nums[0]
    for i in range(1, len(nums)):
        dp[i] = max(nums[i], dp[i-1] + nums[i])
    return max(dp)
```

The defining feature vs Pattern 2: there's no "skip" option — you're always including `nums[i]`, the only decision is whether to *carry forward* the previous sum or *reset*.

Pattern 4 — LIS-style (compare against ALL previous indices)

Recurrence: $dp[i] = \max(dp[j] + 1)$ for all $j < i$ where $condition(j, i)$ holds

Trigger words: "longest increasing subsequence," "longest valid chain," anything where the relevant lookback isn't just 1-2 steps back, but potentially *any* earlier index.

Example: Longest Increasing Subsequence — for each index i , check every earlier index j ; if $nums[j] < nums[i]$, then i could extend the subsequence ending at j .

```
def length_of_lis(nums):
    dp = [1] * len(nums) # every element is a subsequence of length 1 by itself
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
```

This is the one that jumps from $O(n)$ to $O(n^2)$, because instead of looking back at a fixed small window, you're scanning the entire prefix at every step.

What to Actually Write Tonight

1. Fibonacci: $dp[i] = dp[i-1] + dp[i-2]$ | "number of ways"
2. Linear Decision: $dp[i] = \max(dp[i-1], dp[i-2] + x)$ | "can't pick adjacent"
3. Kadane: $dp[i] = \max(x, dp[i-1] + x)$ | "max subarray sum"
4. LIS-style: $dp[i] = \max(dp[j] + 1)$ for $j < i$ | "longest increasing chain"

The Fast Mental Triage in an Interview

Does the answer only depend on a **fixed small window** (last 1-2 states)? → Pattern 1 or 2. Is there a **binary include/exclude decision** with a constraint? → Pattern 2. Are you tracking a **running sum/value that can reset**? → Pattern 3. Do you need to compare against **every earlier index**, not just nearby ones? → Pattern 4 (and expect $O(n^2)$ unless you can optimize with binary search). That last distinction — window-based vs full-prefix-scan — is the difference between $O(n)$ and $O(n^2)$, and interviewers specifically probe for whether you recognize it.

11. 2D Dynamic Programming

Good-to-know

Pattern 1 — Sequence Alignment

Logic: $dp[i][j]$ compares the first i characters of one sequence against the first j of another.

Trigger: two strings, "common subsequence/substring," edit-distance-style.

Example: Longest Common Subsequence.

```
if A[i-1] == B[j-1]:
    dp[i][j] = dp[i-1][j-1] + 1
else:
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Pattern 2 — Grid Path Counting

Logic: $dp[i][j]$ accumulates from valid neighboring cells (above/left, or all 4 directions depending on movement rules).

Trigger: "number of ways to reach the bottom-right of a grid."

Example: Unique Paths.

$dp[i][j] = dp[i-1][j] + dp[i][j-1]$

What to Write Tonight

1. Sequence alignment: $dp[i][j]$ compares two strings | "common subsequence between A and B"
2. Grid path counting: $dp[i][j]$ sums from neighbors | "ways to reach a cell"

Fast Mental Triage

Two distinct sequences being compared → Pattern 1. Single grid, counting paths/ways → Pattern 2.

12. Backtracking 🟡 Important

Pattern 1 — Subset/Combination Building

Logic: At each element, choose to include it or not; recurse either way.

Trigger: "all subsets," "all combinations that sum to target."

Example: Combination Sum.

```
def backtrack(i, path, remaining):
    if remaining == 0: results.append(path[:]); return
    if i >= len(candidates) or remaining < 0: return
    path.append(candidates[i])
    backtrack(i, path, remaining - candidates[i]) # reuse allowed
    path.pop()
    backtrack(i + 1, path, remaining)             # skip
```

Pattern 2 — Permutation Building

Logic: Track which elements are already used; at each position, try every unused element.

Trigger: "all permutations/orderings."

```
def backtrack(path, used):
```

```

if len(path) == len(nums): results.append(path[:]); return
for i, n in enumerate(nums):
    if not used[i]:
        used[i] = True
        path.append(n)
        backtrack(path, used)
        path.pop()
        used[i] = False

```

Pattern 3 — Grid Path Search

Logic: DFS from a cell, marking visited in-place; unmark (backtrack) on dead ends.

Trigger: word/path search on a 2D grid.

Example: Word Search; Word Search II (same DFS, walked against a Trie).

What to Write Tonight

1. Subset/combo: include-or-skip at each element | "all subsets/combinations"
2. Permutation: used-tracking, try every remaining | "all orderings"
3. Grid path: DFS + mark/unmark in place | word/path search on a grid

Fast Mental Triage

Order doesn't matter, just which elements are chosen → Pattern 1. Order matters → Pattern 2.
 2. It's a 2D grid → Pattern 3.

13. Heap / Top-K 🟡 Important

Pattern 1 — Single Size-K Heap

Logic: Maintain a heap capped at size K, evicting the worst element when it overflows.

Trigger: "K largest/smallest/closest."

Example: Top K Frequent Elements; K Closest Points to Origin.

```

import heapq
heap = []
for item in items:
    heapq.heappush(heap, (priority(item), item))
    if len(heap) > k:
        heapq.heappop(heap)

```

Pattern 2 — Two Balanced Heaps

Logic: A max-heap for the lower half and a min-heap for the upper half, kept within 1 of each other in size.

Trigger: "running median," "median of a stream."

Example: Find Median from Data Stream.

Pattern 3 — K-Way Merge

Logic: Push the current head of each of K sequences into a heap; repeatedly pop the smallest, push its successor.

Trigger: "merge K sorted lists/arrays."

Example: Merge K Sorted Lists.

Pattern 4 — Greedy Heap Simulation

Logic: Repeatedly pull the highest-priority item, apply a side effect (cooldown, decrement), and push back if still relevant.

Trigger: scheduling with a recurring constraint.

Example: Task Scheduler — always schedule the most frequent remaining task next.

Note: Sliding Window Maximum is sometimes solved with a heap, but the more standard $O(n)$ solution is actually a **monotonic deque** — see Pattern 2 of the Stack-Based section below. Worth knowing both exist for that problem.

What to Write Tonight

- | | | |
|-----------------|---------------------------------|------------------------------|
| 1. Size-K heap: | cap heap at K, evict worst | "K largest/smallest/closest" |
| 2. Two heaps: | balanced max-heap + min-heap | "running median" |
| 3. K-way merge: | heap of K current heads | "merge K sorted" |
| 4. Greedy sim: | pop highest priority repeatedly | scheduling w/ cooldown |

Fast Mental Triage

"K" + largest/smallest/closest → Pattern 1. "Median" of a changing stream → Pattern 2. Multiple already-sorted things to combine → Pattern 3. Repeated greedy choice from a changing priority set → Pattern 4.

14. Stack-Based Patterns Important

Pattern 1 — Matching/Resolution Stack

Logic: A stack models "the most recently opened thing must be resolved first" — last-in-first-out resolution order.

Trigger words: "matching pairs," "resolve against the most recent unresolved item," expression evaluation.

Examples:

- **Valid Parentheses** — push openers; on a closer, pop and check it matches; valid iff stack is empty at the end.
- **Asteroid Collision** — push right-movers; on a left-mover, resolve collisions against the stack top repeatedly before deciding whether to push.
- **Basic Calculator II** — stack of pending numbers + last-seen operator, to correctly prioritize $*///$ over $+/-$.

```
def is_valid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        else:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

Pattern 2 — Monotonic Stack/Queue

Logic: Maintain a stack (or deque) that's always increasing or always decreasing in value. Before pushing a new element, pop everything that would violate that order — those popped elements have just found their answer (the new element is their "next greater/smaller").

Trigger words: "next greater/smaller element," "daily temperatures," sliding window maximum/minimum, "how many days until..."

Examples:

- **Daily Temperatures** — maintain a decreasing stack of indices (by temperature). When a warmer day arrives, pop everyone it's warmer than, recording the gap for each.

- **Sliding Window Maximum** — maintain a decreasing deque of indices; pop from the back when a larger element arrives (they can never be the max again), pop from the front when an index slides out of the window. The front of the deque is always the current max — this is the $O(n)$ alternative to the size-K heap approach.
- **Car Fleet** — sort by position, then a monotonic stack of arrival times tracks which cars merge into the same fleet.

```
stack = [] # holds indices, kept monotonic by value
for i, val in enumerate(arr):
    while stack and arr[stack[-1]] < val: # decreasing stack example
        j = stack.pop()
        answer[j] = i - j # j just found its "next greater" at index i
    stack.append(i)
```

The defining tell versus Pattern 1: there's no explicit "opening/closing" symbol to match — instead, you're discovering each element's relationship to the *next* element that breaks the current trend, and the stack/deque only ever holds elements still "waiting" for that answer.

Write tonight

1. Matching: push openers, pop+match on closers | "matching pairs" / expression eval
2. Monotonic: keep stack increasing/decreasing, | "next greater/smaller" /
pop violators before pushing new | sliding window max/min

Triage

Undoing or resolving against "whatever came most recently" with explicit open/close semantics → Pattern 1. "Next greater/smaller element," or needing the max/min of a *sliding* range efficiently → Pattern 2 (and remember: this is the $O(n)$ upgrade over a heap for Sliding Window Maximum specifically).

15. Matrix Traversal / In-Place Manipulation Good-to-know

Pattern 1 — Boundary-Shrinking Traversal

Logic: Track top/bottom/left/right boundaries; traverse each side, then shrink inward.

Trigger: "spiral order."

Example: Spiral Matrix.

```
top, bottom, left, right = 0, rows-1, 0, cols-1
while top <= bottom and left <= right:
```

```

for c in range(left, right+1): visit(top, c)
top += 1
for r in range(top, bottom+1): visit(r, right)
right -= 1
if top <= bottom:
    for c in range(right, left-1, -1): visit(bottom, c)
    bottom -= 1
if left <= right:
    for r in range(bottom, top-1, -1): visit(r, left)
    left -= 1

```

Pattern 2 — Transpose + Reverse / In-Place Marker Reuse

Logic: Decompose a transform into simpler known steps, or reuse the matrix's own edges as scratch space.

Trigger: "rotate the matrix," "without extra space."

Example: Rotate Image (transpose then reverse each row); Set Matrix Zeroes (first row/column as markers).

What to Write Tonight

1. Boundary-shrink: 4 shrinking edges, traverse each | "spiral order"
 2. Transpose/reuse: decompose into known steps, or reuse | "rotate" / "in-place, no extra space"
- the grid's own edges as scratch space

Fast Mental Triage

"Spiral" → boundary-shrinking. "Rotate" or "in-place with no extra space" → look for a known 2-step decomposition or reusable scratch space within the grid itself.

16. Bit Manipulation Good-to-know

Logic: A toolbox of bitwise tricks (AND, OR, XOR, shifts) that solve numeric problems in $O(1)$ per operation. This is genuinely a toolbox, not a single pattern with variants — there's nothing structurally different between the examples beyond which trick applies.

Trigger words: "without +/-," counting bits, anything explicitly about binary representation.

Examples:

- **Counting Bits** — $dp[i] = dp[i \gg 1] + (i \& 1)$.
- **Reverse Bits** — shift result left, OR in n 's lowest bit, shift n right, repeat 32 times.
- **Missing Number** — XOR every index with every value; paired entries cancel, leaving the unpaired missing number.
- **Sum of Two Integers** — XOR gives the sum without carrying; $(a \& b) \ll 1$ gives exactly the carry; repeat until carry is 0.
- **Number of 1 Bits** — $n \& (n-1)$ clears the lowest set bit each iteration.

```
def hammingWeight(n):
```

```
    count = 0
    while n:
        n &= (n - 1)
        count += 1
    return count
```

Write tonight

Bit Manip: $n \& 1$ (check bit), $n \& (n-1)$ (clear lowest), XOR (cancel pairs) | "binary representation"

Triage

XOR's self-canceling property is the most reused trick, $n \& (n-1)$ is the second most. This is more toolbox than single pattern.

17. Design a Data Structure

Must-know

This isn't an algorithmic *technique* like the other 16 — it's a recurring **interview format**: build a class supporting specific operations, each under a stated time complexity. The skill is picking the right *combination* of structures so every operation hits its target simultaneously, usually trading a little memory for speed. The pattern to internalize: **a HashMap almost always handles the $O(1)$ lookup half of the requirement — the real design decision is what you pair it with to handle ordering, eviction, or ranking.**

Pattern 1 — HashMap + Doubly Linked List (order by recency)

Logic: HashMap gives $O(1)$ access to any node by key; a doubly linked list maintains usage order, so the most/least recently used item is always at a known end. Every access moves a node to the front; eviction removes from the back.

Trigger: "least recently used," fixed-capacity cache with $O(1)$ get/put.

Example: LRU Cache.

```
class Node:
    def __init__(self, key, val):
        self.key, self.val = key, val
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.head, self.tail = Node(0, 0), Node(0, 0)
        self.head.next, self.tail.prev = self.tail, self.head

    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _insert_front(self, node):
        node.next, node.prev = self.head.next, self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.map: return -1
        node = self.map[key]
        self._remove(node)
        self._insert_front(node)
        return node.val

    def put(self, key, val):
        if key in self.map:
            self._remove(self.map[key])
        node = Node(key, val)
        self.map[key] = node
        self._insert_front(node)
        if len(self.map) > self.cap:
            lru = self.tail.prev
            self._remove(lru)
            del self.map[lru.key]
```

Pattern 2 — HashMap + Frequency Buckets (order by count)

Logic: Same idea as Pattern 1, but the secondary ordering is by *access frequency* instead of recency. A HashMap maps each frequency count to its own doubly linked list of keys at that frequency; eviction always pulls from the lowest-frequency bucket.

Trigger: "least frequently used," eviction by access count rather than recency.

Example: LFU Cache.

The implementation is meaningfully more involved than LRU (tracking `min_freq` as it changes, moving nodes between buckets on every access) — for interview purposes, the important thing to be able to say out loud is *why* a single linked list isn't enough: recency and frequency are different orderings, so you need a structure that can promote a key from one frequency bucket to the next every time it's accessed, while still supporting $O(1)$ eviction from whichever bucket is currently lowest.

Pattern 3 — HashMap + Swap-and-Pop Array ($O(1)$ arbitrary removal)

Logic: Arrays give $O(1)$ random access but $O(n)$ removal from the middle (everything has to shift). The fix: swap the element you want to remove with the *last* element, then pop the last element — removal becomes $O(1)$ because you never need to preserve order.

Trigger: "insert, remove, and get a random element, all $O(1)$."

Example: Insert Delete GetRandom $O(1)$.

```
import random
```

```
class RandomizedSet:
    def __init__(self):
        self.arr = []
        self.idx = {} # value -> index in arr

    def insert(self, val):
        if val in self.idx: return False
        self.idx[val] = len(self.arr)
        self.arr.append(val)
        return True

    def remove(self, val):
        if val not in self.idx: return False
        last_val = self.arr[-1]
        i = self.idx[val]
        self.arr[i] = last_val
        self.idx[last_val] = i
        self.arr.pop()
```

```

        self.arr[i] = last_val      # move last element into the gap
        self.idx[last_val] = i
        self.arr.pop()
        del self.idx[val]
        return True

def getRandom(self):
    return random.choice(self.arr)

```

Pattern 4 — HashMap + Sorted Append List + Binary Search (order by time)

Logic: Values for a given key are only ever appended with increasing timestamps, so the list is automatically sorted with zero extra work. A query for "value at time T" becomes a binary search for the largest stored timestamp $\leq T$.

Trigger: versioned/timestamped key-value storage, "get the value as of time T."

Example: Time Based Key-Value Store.

```
import bisect
```

```

class TimeMap:
    def __init__(self):
        self.store = {} # key -> list of (timestamp, value)

    def set(self, key, value, timestamp):
        self.store.setdefault(key, []).append((timestamp, value))

    def get(self, key, timestamp):
        if key not in self.store: return ""
        arr = self.store[key]
        i = bisect.bisect_right(arr, (timestamp, chr(0x10FFFF)))
        return arr[i-1][1] if i > 0 else ""

```

Combination callout — Design Twitter

Design Twitter doesn't need a 5th pattern — it's Pattern 3's storage style (HashMap of user → list of tweets) combined directly with the **Heap K-way merge** pattern from category 13, used here to merge each followee's tweets into one ranked, time-ordered feed. Recognizing that a "design" question is really *two* patterns stacked together is itself the skill this category is training — most of these problems are combinations once you've internalized the building blocks individually.

What to Write Tonight

1. LRU: HashMap + doubly linked list | order by recency
 2. LFU: HashMap + frequency-bucketed lists | order by access count
 3. O(1) remove: HashMap + array, swap-with-last-then-pop | random access + O(1) delete
 4. Time-based: HashMap + append-only list + binary search | order by timestamp
- + Combinations: storage pattern + a retrieval pattern | e.g. Design Twitter = storage + heap merge
from elsewhere in this document

Fast Mental Triage

Eviction by "used most recently"? → Pattern 1. Eviction by "used most often"? → Pattern 2. Need O(1) removal from an unordered collection? → Pattern 3 (swap-and-pop is the trick people forget). Querying by time/version? → Pattern 4. If the problem seems to need *two* behaviors at once (storage + ranking, storage + merging) — that's usually two patterns from this document stacked, not a new one.

Closing Note

That's the complete map — 17 categories, nothing in Blind 75 or your Google-tagged revision pool sits outside it. If a future problem doesn't immediately match one of these, the right move in an interview is to say so out loud and reason from first principles rather than forcing a fit — but across the actual practice you've done so far, every single problem has landed cleanly into one of these 17, which is exactly why this list is worth knowing cold rather than re-deriving from scratch under pressure.